

Final Project: Robust Neural Decoding under Real-World Constraints

Objective

In your mini project, you demonstrated that modern machine learning models (Neural Networks) can outperform classical methods (Wiener Filter) for decoding behavior from neural activity.

In this final project, you will extend this work to a more realistic and challenging setting.

Your goal is to develop and analyze neural decoding models under **real-world constraints**, including:

- neural drift (changes over time)
- missing input channels (channel dropout)

This project focuses not only on performance, but also on **robustness and generalization**, which are critical in practical brain-computer interface (BCI) systems.

Dataset

You will use: <https://www.kaggle.com/competitions/intracortical-neural-activity-decoding-phase-2-cs-gy-9223/overview>

Dataset Description

- 96-channel neural recordings (Spiking Band Power)
- Time-series data collected across multiple sessions
- Target variables:
 - finger position (continuous values in $[0, 1]$)
- Challenges:
 - ~30% of channels are randomly missing per session
 - training and test sessions are separated in time (neural drift)

This is a **regression problem** (same framework as your mini project)

Reference Materials

- Machine Learning for Neural Decoding
- Neural_Decoding GitHub Repository

Scientific Questions

Your project should address at least one of the following:

- Why do classical models (e.g., Wiener Filter) fail under real-world conditions?
- How do neural networks handle missing input channels?
- How does neural drift affect decoding performance?
- How can we improve robustness of neural decoders across sessions?

Project Tasks

Step 1 : Data Understanding

- Load and explore the dataset
- Identify:
 - input features (neural activity)
 - output variables (finger positions)

Provide a short description of:

- data structure
- challenges (missing channels, session variability)

Step 2 : Baseline Implementation

Implement at least one baseline model:

- Wiener Filter (linear regression)

Evaluate using:

- R^2 score

Step 3 : Neural Network Model

Implement a neural network model:

- Feedforward Neural Network or RNN (optional)

Train and evaluate using the same setup as the baseline.

Step 4 : Core Experiment (Required)

You must perform **at least one** of the following experiments:

Option A : Channel Dropout Robustness

- Analyze model performance with missing channels
- Compare:
 - baseline vs neural network

Option B : Neural Drift Analysis

- Train on earlier sessions
- test on later sessions
- analyze performance degradation

Option C : Velocity-Based Decoding (Advanced)

- Predict velocity instead of position
- reconstruct position via integration
- compare performance

Step 5 : Evaluation

- Compute R^2 for all models
- Compare results across experiments

Step 6 : Analysis (MOST IMPORTANT)

Answer clearly:

- Which model performs best and why?
- How does performance change under constraints?
- What are the main challenges in this dataset?
- What strategies improve robustness?

Expected Outputs

Results Tables

- Model performance (R^2 scores)

Figures

- Predicted vs true signal (example)
- Performance comparison plots

Final Report (3–5 pages)

Include:

1. Introduction

- Problem description
- Motivation

Brain-computer interface (BCI) systems aim to translate neural activity into meaningful behavioral outputs, enabling applications such as prosthetic control, assistive technologies, and neurorehabilitation. A key challenge in this domain is accurately decoding continuous motor signals from high-dimensional neural recordings, which are often noisy and complex. Traditional approaches, such as linear models, assume simple relationships between neural activity and movement, but real neural systems exhibit nonlinear and distributed patterns that are difficult to capture with such methods.

In real-world settings, an additional challenge arises due to missing or unreliable input channels caused by sensor degradation, noise, or hardware limitations. This can significantly degrade model performance and limits the reliability of BCI systems outside controlled environments. Therefore, robustness to missing input information becomes an essential requirement for practical neural decoding systems.

This project focuses on analyzing how neural networks handle missing input channels in comparison to classical models such as the Wiener Filter. Using ECoG-based finger movement data, the study evaluates model performance under simulated channel dropout conditions, with the aim of understanding how different approaches generalize under incomplete input scenarios. To make the evaluation more reliable, the experiment was repeated across three subject files from the BCI Competition IV Dataset 4, allowing the analysis to compare model behavior across different neural recordings rather than relying on a single subject.

2. Data Understanding

The dataset used in this study is the BCI Competition IV Dataset 4, which contains electrocorticography (ECoG) recordings collected during individual finger movement tasks. The data consists of multi-channel neural signals recorded from 62 electrodes, along with corresponding continuous finger flexion measurements for five fingers. Each sample represents a time step, making the dataset inherently temporal, with a total of 400,000 training samples and 200,000 test samples. The analysis was performed separately on three subject files, referred to as S1, S2, and S3. Each subject file contains neural input recordings and corresponding five-finger flexion outputs, but the number of channels varies slightly across subjects. This subject-wise setup is useful because it allows model performance to be evaluated across different neural recording conditions.

The input features correspond to neural activity across multiple channels, while the output variables represent continuous finger positions. This establishes the problem as a multi-output regression task, where the goal is to learn a mapping from neural signals to motor behavior. Figure

1 illustrates example finger trajectories over time, highlighting the continuous and dynamic nature of the target signals. In contrast, Figure 2 shows a subset of neural channels, demonstrating the variability and noise present in the input data.

A key challenge in this dataset is the high dimensionality of the input space combined with the complex relationship between neural activity and motor output. Additionally, although the dataset itself does not contain missing values, real-world scenarios often involve incomplete channel information. This motivates the need to simulate missing channels during experimentation to evaluate model robustness.

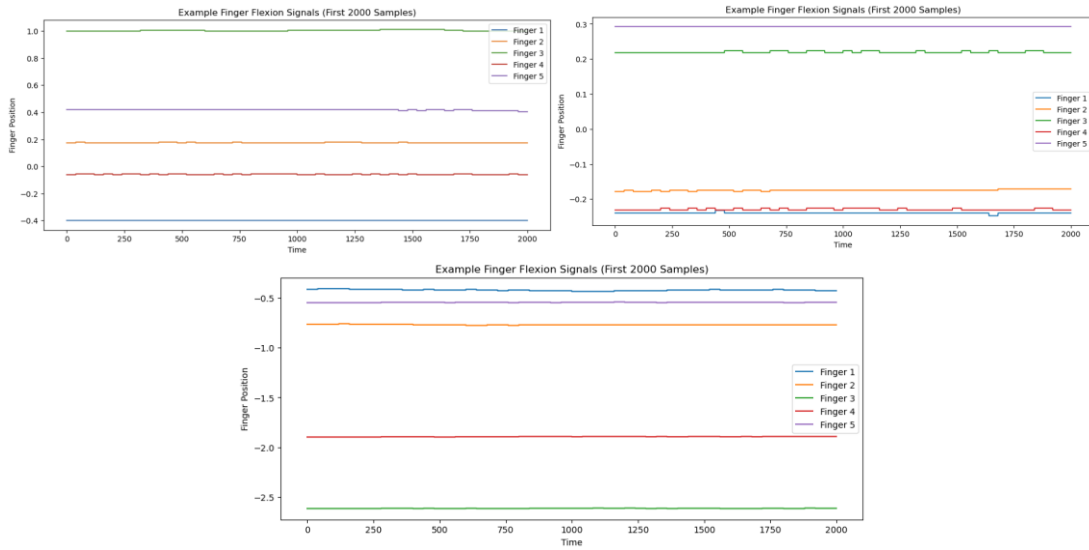


Figure 1: Example Finger Flexion Signals (First 2000 Samples for S1, S2 and S3)

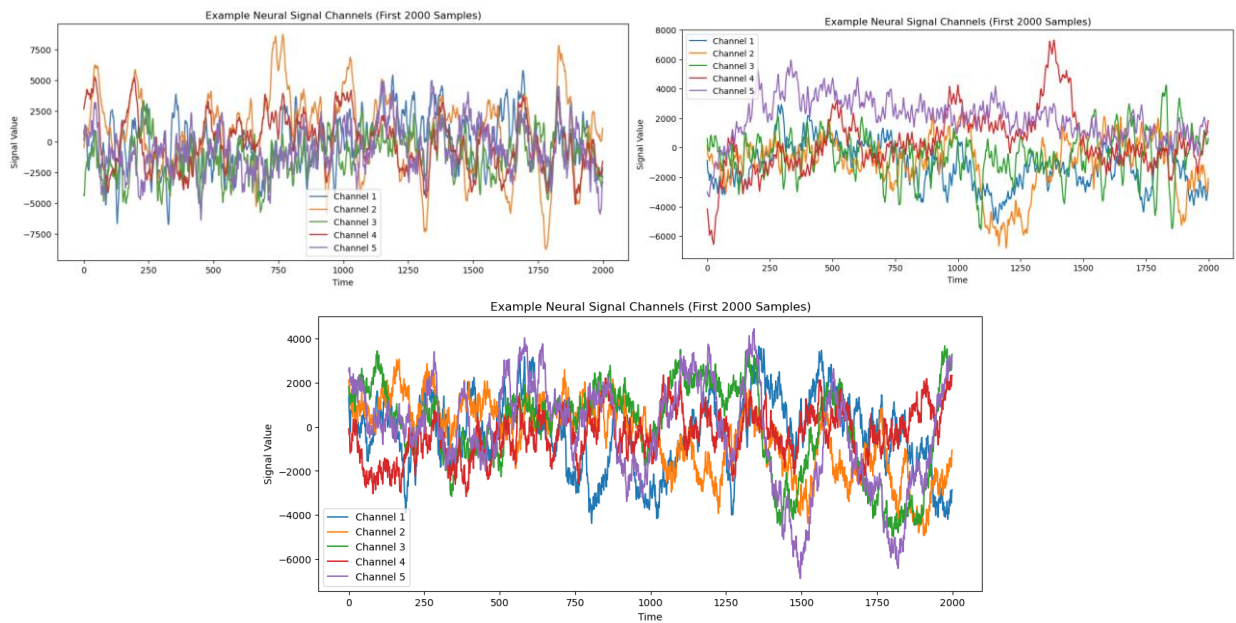


Figure 2: Example Neural Signal Channels (First 2000 Samples for S1, S2 and S3)

3. Methodology

- Models
- Experimental setup

To evaluate model performance, both classical and neural approaches are considered. The Wiener Filter is implemented using linear regression as a baseline model. This method assumes a linear relationship between neural inputs and finger movements and serves as a simple and interpretable reference point. However, due to its linear nature, it is expected to struggle with capturing the nonlinear dynamics of neural activity.

To address these limitations, a feedforward neural network is employed. The network consists of multiple dense layers with nonlinear activation functions, allowing it to model complex relationships between input signals and output variables. Dropout layers are included to improve generalization and reduce overfitting. The model is trained using mean squared error loss and optimized using the Adam optimizer.

To ensure fair comparison, both models are trained on standardized input data, where each feature is normalized to have zero mean and unit variance. The dataset is split into training and validation sets to evaluate performance during development. To evaluate robustness, missing input channels were simulated by randomly removing a fixed percentage of input features during testing. Specifically, 30% of the neural channels were set to zero, and model performance was evaluated under this condition to analyze the impact of incomplete input data. The same training, validation, and channel-dropout procedure was repeated independently for S1, S2, and S3. In addition to the 30% dropout condition, robustness was further evaluated across increasing dropout levels of 0%, 10%, 30%, 50%, and 70%. This provides a clearer view of how model performance changes as more neural channels become unavailable.

4. Results

- Tables and figures

The performance of the Wiener Filter and the neural network model is evaluated using the coefficient of determination (R^2 score). Table 1 presents the results for both models under normal conditions, as well as under simulated missing-channel scenarios where 30% of the input channels are randomly removed. The relative reduction in R^2 score is significantly larger for the Wiener Filter compared to the neural network, indicating that classical models are more sensitive to incomplete input information.

Table 1: Comparison of Wiener Filter and NN R^2 for Original and 30% Dropout

Subject	Wiener Filter R^2	Neural Network R^2	Wiener Filter R^2 at 30% Dropout	Neural Network R^2 at 30% Dropout
S1	0.032	0.491	0.011	0.104
S2	0.038	0.682	0.027	0.238
S3	0.085	0.593	0.059	0.258
Average	0.052	0.588	0.032	0.200

The subject-wise results show that the neural network consistently outperformed the Wiener Filter across all three subject files. Under normal conditions, the Wiener Filter achieved low R^2 scores ranging from 0.032 to 0.085, while the neural network achieved substantially higher R^2 scores ranging from 0.491 to 0.682. This confirms that the neural network is better able to capture the nonlinear relationship between ECoG activity and finger movement. Under 30% channel dropout, both models experienced performance degradation, but the neural network still maintained higher predictive performance than the Wiener Filter for every subject. This consistent trend across all subjects indicates that the superiority of neural networks is not dependent on a specific dataset but reflects a general advantage in modeling neural decoding tasks.

Table 2: Model robustness across increasing channel-dropout levels for all three subjects.

Dropout Level	S1 Wiener R^2	S1 NN R^2	S2 Wiener R^2	S2 NN R^2	S3 Wiener R^2	S3 NN R^2
0%	0.032	0.491	0.038	0.682	0.085	0.593
10%	0.027	0.285	0.037	0.513	0.079	0.452
30%	0.011	0.104	0.027	0.238	0.059	0.258
50%	0.007	0.006	0.013	0.029	0.045	0.091
70%	0.009	-0.052	-0.001	-0.276	0.023	-0.188

Table 2 shows a consistent decrease in R^2 score as the percentage of missing channels increases. The neural network performs best when complete or moderately incomplete neural data is available, but its performance drops sharply under severe dropout conditions such as 50% and 70%. Negative R^2 values at 70% dropout indicate that the model performs worse than predicting the average target value, showing that extreme channel loss removes too much information for reliable decoding. This trend clearly shows that while neural networks provide strong performance under normal conditions, their robustness is limited under extreme channel loss, highlighting the importance of designing models specifically for missing-data scenarios.

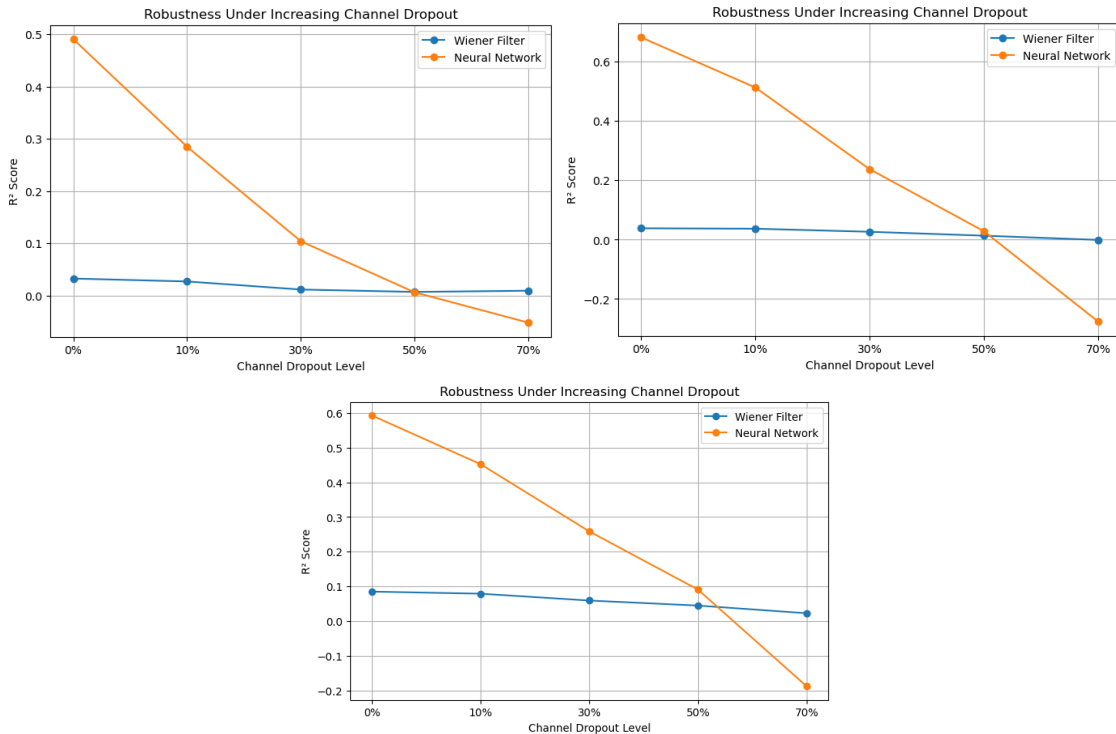


Figure 3: Robustness under increasing channel dropout (for S1, S2 and S3)

To further analyze the impact of missing channels, model performance is evaluated after randomly dropping 30% of the input features. As shown in Table 1, the Wiener Filter experiences a sharp decline in performance, while the neural network, although affected, maintains a substantially higher R^2 score. **Figure 3** shows that neural network performance is strong when all channels are available, but it decreases sharply as more channels are removed. The Wiener Filter remains low across all dropout levels, while the neural network provides better performance under moderate dropout but becomes unreliable under severe dropout.

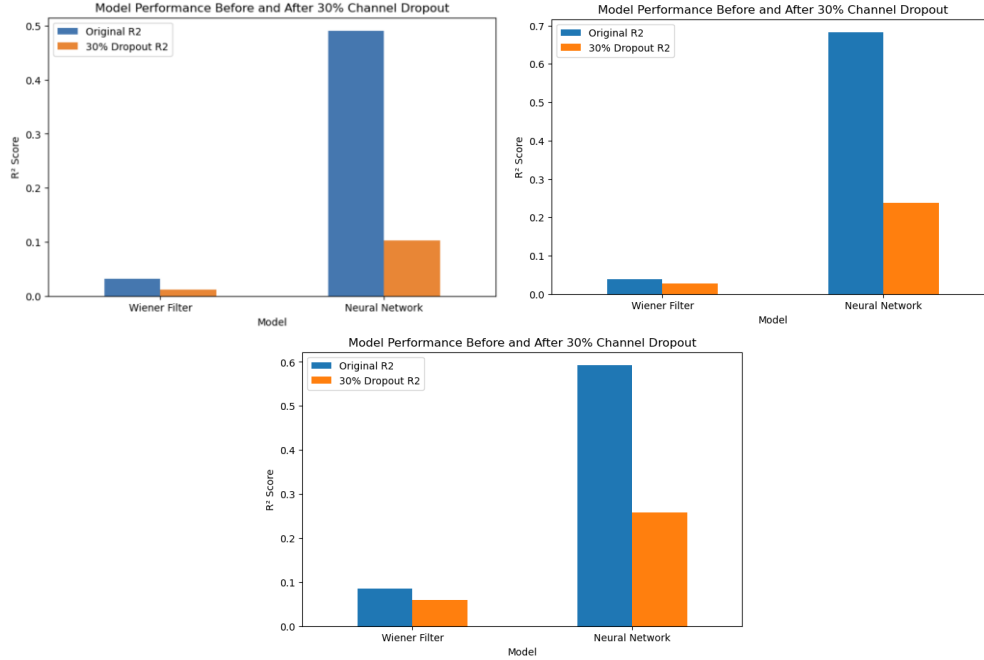


Figure 4: Model Performance Before and After 30% Channel Dropout (for S1, S2 and S3)

Table 3: Comparison Per Finger decoding performance, R^2 (for S1, S2 and S3)

Finger	S1 (R^2)		S2 (R^2)		S3 (R^2)	
	Wiener	NN	Wiener	NN	Wiener	NN
F1	0.027	0.429	0.041	0.682	0.080	0.674
F2	0.028	0.483	0.045	0.656	0.128	0.591
F3	0.048	0.546	0.046	0.752	0.066	0.493
F4	0.033	0.491	0.040	0.645	0.095	0.580
F5	0.026	0.504	0.020	0.678	0.056	0.625

Table 3 shows the per-finger decoding performance for all three subject files. The neural network outperforms the Wiener Filter for every finger across S1, S2, and S3, confirming that the performance advantage is not limited to one output variable. S2 shows the strongest per-finger neural network performance overall, while S1 is more difficult to decode. This supports the observation that decoding performance varies across subjects, but the neural network remains consistently stronger than the Wiener Filter. The results demonstrate that neural network performance is consistently higher across all fingers and subjects, indicating that the model's advantage is stable across different output variables and not limited to overall averages.

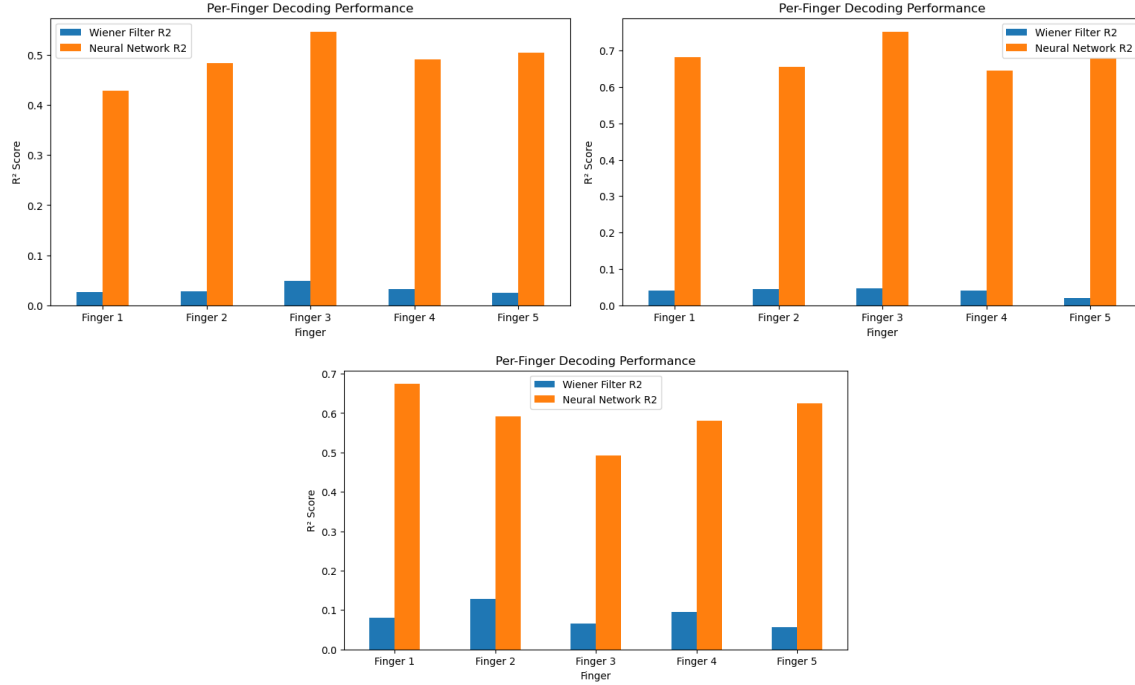


Figure 5: Plot of Per Finger Decoding Performance (for S1, S2 and S3)

5. Discussion

- Interpretation of results
- Challenges and limitations

Across all three subject files, the neural network consistently achieved stronger decoding performance than the Wiener Filter. The average R^2 score of the neural network under normal conditions was approximately 0.588, compared to only 0.052 for the Wiener Filter. This indicates that the nonlinear neural network model was much more effective at learning the relationship between neural activity and finger movement. Under 30% channel dropout, both models degraded, but the neural network still maintained an average R^2 score of approximately 0.200, while the Wiener Filter dropped to 0.032.

The observed behavior can be attributed to the fundamental differences between linear and nonlinear models. The Wiener Filter relies on a fixed linear mapping between input and output, making it highly sensitive to the loss of input features. In contrast, neural networks learn distributed representations, where information is encoded across multiple channels rather than relying on individual inputs. This allows the model to partially compensate for missing channels, resulting in improved robustness. Additionally, the nonlinear activation functions in neural networks enable them to capture complex relationships in neural data, which further contributes to their superior performance.

The results also show subject-to-subject variability. For example, S2 achieved the strongest neural network performance under normal conditions, while S1 showed the lowest neural network performance. This suggests that decoding difficulty varies across neural recordings, possibly due to differences in signal quality, channel configuration, or subject-specific neural patterns. However, the overall trend remained consistent: neural networks performed better than the Wiener Filter across all subjects and all moderate dropout conditions.

Despite this advantage, the results also indicate that missing channels negatively impact overall decoding accuracy, suggesting that even neural networks rely heavily on complete input information. This highlights the importance of developing strategies to handle incomplete neural data in practical BCI systems.

The main challenges in this dataset include the high dimensionality of neural inputs, the complex and nonlinear relationship between neural activity and motor output, and the sensitivity of models to incomplete input information. Although the dataset itself does not contain missing values, the simulation of channel dropout highlights how real-world conditions can significantly impact model performance. Additionally, the simulation of missing channels highlights the limitations of models in real-world scenarios where data may be noisy or partially unavailable. These findings directly answer the scientific question by demonstrating that neural networks handle missing input channels more effectively than classical linear models, particularly under moderate levels of channel dropout.

6. Conclusion

- Key findings
- Future work

This study demonstrates that neural networks outperform classical linear models in decoding finger movements from ECoG signals across multiple subject files. Across S1, S2, and S3, the neural network achieved substantially higher R^2 scores than the Wiener Filter under normal conditions and maintained stronger performance under 30% channel dropout. These results directly answer the scientific question by showing that neural networks handle missing input channels more effectively than classical models, although severe dropout conditions still significantly reduce performance.

Several strategies can improve robustness in neural decoding systems. These include the use of nonlinear models such as neural networks, which can better capture complex patterns, as well as techniques such as dropout regularization, data augmentation, and training with simulated missing channels. These approaches enable the model to generalize better and maintain performance even when input data is incomplete.

Future work could explore additional strategies to improve robustness, such as training models with varying levels of channel dropout, incorporating temporal models such as recurrent neural

networks, and evaluating performance across multiple subjects or sessions. These approaches may further enhance generalization and reliability in real-world neural decoding systems.

Evaluation Criteria

Your project will be evaluated based on:

- Correct implementation of models
- Quality of experiments
- Depth of analysis
- Clarity of explanations
- Ability to connect results to scientific questions

Q1. Which model performs best and why?

Answer: The neural network performs best across all experiments and subjects. Under normal conditions, the neural network achieves significantly higher R^2 scores compared to the Wiener Filter, with an average performance of approximately 0.588 compared to 0.052 for the Wiener Filter. This difference is consistent across all three subject datasets. The superior performance of the neural network can be attributed to its ability to model nonlinear relationships between neural activity and finger movement. Unlike the Wiener Filter, which assumes a fixed linear mapping, the neural network learns complex patterns and distributed representations across multiple channels. This allows it to capture subtle variations in neural signals that are essential for accurate decoding.

Q2. How does performance change under constraints?

Answer: Model performance decreases as the level of channel dropout increases. For both models, R^2 scores decline steadily from 0% to 70% dropout, indicating that missing input channels significantly impact decoding accuracy. However, the rate of degradation differs between the two models. The Wiener Filter shows consistently low performance and degrades gradually, while the neural network initially maintains strong performance under moderate dropout (10%-30%) but experiences a sharp decline at higher levels (50%-70%). At 70% dropout, the neural network produces negative R^2 values, indicating that it performs worse than predicting the mean output. This shows that while neural networks are more robust under moderate constraints, both models struggle under extreme loss of input information.

Q3. What are the main challenges in this dataset?

Answer: The primary challenges in this dataset include high dimensionality, complex nonlinear relationships, and sensitivity to missing input channels. The neural signals are high-dimensional, with dozens of input channels, making it difficult for simple models to identify meaningful patterns. Additionally, the relationship between neural activity and finger movement is nonlinear and distributed across channels, which limits the effectiveness of linear models. Another major challenge is the presence of missing or unreliable channels, which can significantly reduce model performance. Although the dataset itself does not contain missing values, the simulated channel

dropout highlights how real-world conditions can negatively affect decoding accuracy. Finally, subject-to-subject variability introduces additional complexity, as decoding performance differs across S1, S2, and S3.

Q4. What strategies improve robustness?

Answer: Several strategies can improve robustness in neural decoding models. First, using nonlinear models such as neural networks significantly improves performance under both normal and constrained conditions. Second, incorporating dropout during training helps the model learn to handle missing input channels more effectively. Third, training with simulated channel dropout allows the model to generalize better to incomplete data. Additionally, standardizing input features improves training stability and model performance. Other potential improvements include using more advanced architectures such as recurrent neural networks to capture temporal dependencies, and incorporating data augmentation techniques to increase model resilience. These strategies collectively help improve robustness and make neural decoding models more suitable for real-world applications.

Appendix

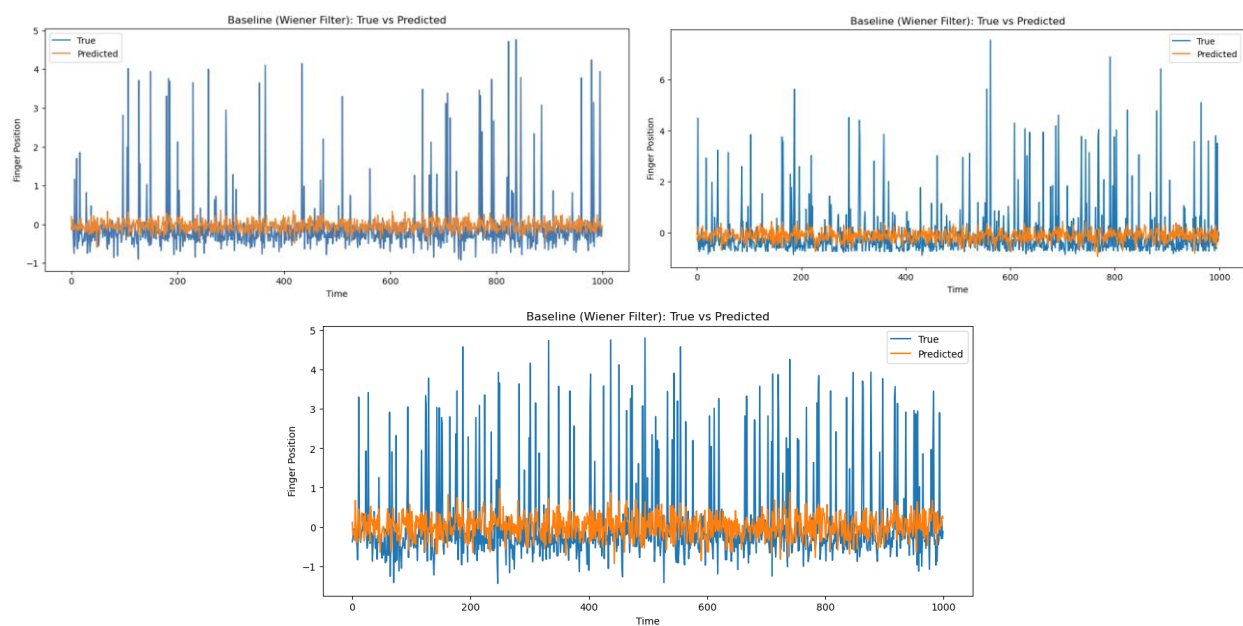


Figure 6: Wiener Filter True Vs Predicted graph (for S1, S2 and S3)

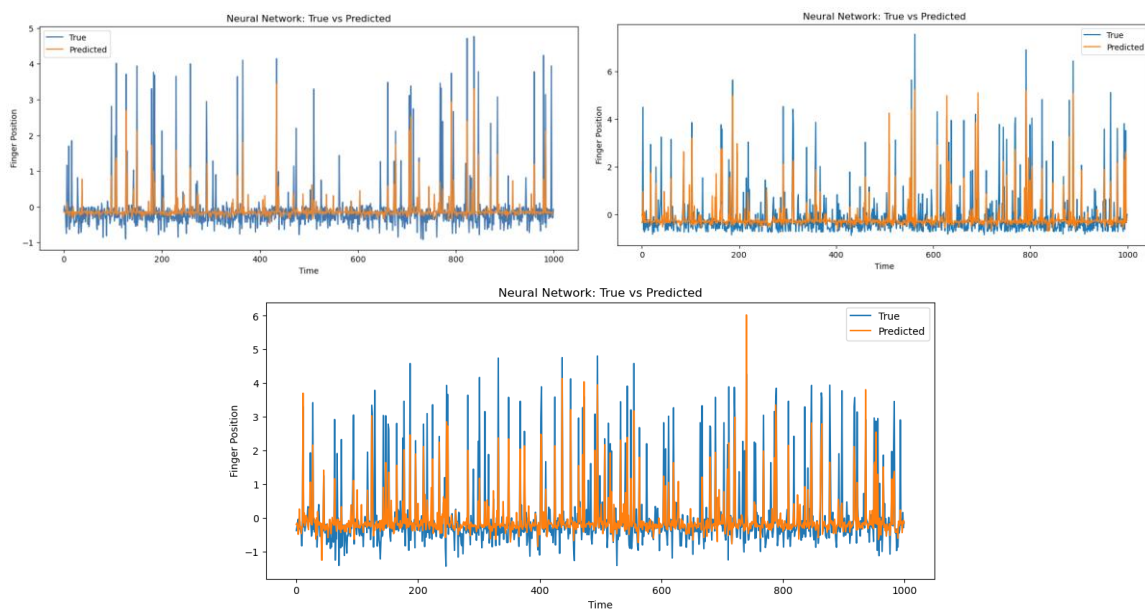


Figure 7: Neural Network True Vs Predicted graph (for S1, S2 and S3)